Lab 1: Access Control Lab – Phase I

CNIT27000-001

Sameer Pirzada

Submitted To: Brian Zuel

Date Submitted: 5/31/2026

Date Due: 5/31/2026

Table of Contents

PROCEDURES ***4***

Creating Users ***5***

Permission Bits Review ***13***

Creating Groups ***15***

Group Audit ***21***

Audit Method – Authorized User Use Cases **21**

Use Case 1 - Create a new file 22

Use Case 2 - Read a file 22

Use Case 3 - Modify a file the user owns 22

Use Case 4 - Delete a file the user owns 23

Use Case 5 - List directory contents 23

Audit Method - Authorized User Mis-Cases **23**

Misuse Case 1 - Delete a file owned by another group member 23

Misuse Case 2 - Delete the entire group share directory 24

Audit Method - Unauthorized User Misuse Cases **24**

Misuse Case 4 - Non-member attempts to list directory contents....24

Misuse Case 5 - Non-member attempts to read a file....24

Misuse Case 6 - Non-member attempts to create a file....25

Misuse Case 7 - Non-member attempts to delete a file....25

Misuse Case 8 - Non-member attempts to delete the share directory....25

Root Account....25

Disable Root Login Method....25

/etc & /etc/shadow....26

Create Character Devices....29

Chroot Jail: Testing....35

Chroot Jail: Making the Jail Nice....36

Chroot Jail: Nice Jail Testing....39

PROCEDURES

Procedures detailed and inserted follow a specific formatting key to ensure professionalism and cleanliness. The formatting key is as follows: **buttons and Windows keys** are bold, *options* are italicized, `text entered into the computer` is in Courier New, menu navigation is by the pipe symbol and italic words: *Start* | *Programs* | *MS Office* | *Word*.

Creating Users

The following section involves the methods to create the users: Alice, Bob, Carol, Eve, Malory, Trent, Walter, Victor, and Hands. The basic syntax for the `adduser` command is `sudo adduser <options> <username>`. This command creates the user with the provided username and first available UID. It then makes a group with the same name and uses the first available GID. Additionally, this command creates a home directory for the user (`/home/<username>`). Finally, it starts the `passwd` command to set up the new users password and asks for additional user details. This was repeated for all created users (Dancuk, 2025). Figures 1-9 provide evidence of the method used to create users.

1. Opened **Terminal**

2. Typed sudo adduser <username>
3. Skipped password prompt by pressing enter
4. Typed <username> for Full Name prompt
5. Skipped Room Number, Work Phone, Home Phone, and Other prompts by pressing enter
6. Typed `yes`
7. Repeated for all users

```
cyberstudent@CNIT270-Summer-2026:~$ sudo adduser alice
info: Adding user `alice' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `alice' (1001) ...
info: Adding new user `alice' (1001) with group `alice (1001)' ...
info: Creating home directory `/home/alice' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: No password supplied
Retype new password:
No password has been supplied.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N]
Changing the user information for alice
Enter the new value, or press ENTER for the default
        Full Name []: Alice
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n]
info: Adding new user `alice' to supplemental / extra groups `users' ...
info: Adding user `alice' to group `users' ...
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 1: Creating user account for Alice

```
cyberstudent@CNIT270-Summer-2026:~$ sudo adduser bob
info: Adding user `bob' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `bob' (1002) ...
info: Adding new user `bob' (1002) with group `bob (1002)' ...
info: Creating home directory `/home/bob' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: No password supplied
Retype new password:
No password has been supplied.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N]
Changing the user information for bob
Enter the new value, or press ENTER for the default
        Full Name []: Bob
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n]
info: Adding new user `bob' to supplemental / extra groups `users' ...
info: Adding user `bob' to group `users' ...
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 2: Creating user account for Bob

```
cyberstudent@CNIT270-Summer-2026:~$ sudo adduser carol
info: Adding user `carol' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `carol' (1003) ...
info: Adding new user `carol' (1003) with group `carol (1003)' ...
info: Creating home directory `/home/carol' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: No password supplied
Retype new password:
No password has been supplied.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N]
Changing the user information for carol
Enter the new value, or press ENTER for the default
        Full Name []: Carol
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n]
info: Adding new user `carol' to supplemental / extra groups `users' ...
info: Adding user `carol' to group `users' ...
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 3: Creating user account for Carol

```
cyberstudent@CNIT270-Summer-2026:~$ sudo adduser eve
[sudo] password for cyberstudent:
info: Adding user `eve' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `eve' (1004) ...
info: Adding new user `eve' (1004) with group `eve (1004)' ...
info: Creating home directory `/home/eve' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: No password supplied
Retype new password:
No password has been supplied.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N]
Changing the user information for eve
Enter the new value, or press ENTER for the default
        Full Name []: Eve
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n]
info: Adding new user `eve' to supplemental / extra groups `users' ...
info: Adding user `eve' to group `users' ...
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 4: Creating user account for Eve

```
cyberstudent@CNIT270-Summer-2026:~$ sudo adduser malory
info: Adding user `malory' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `malory' (1005) ...
info: Adding new user `malory' (1005) with group `malory (1005)' ...
info: Creating home directory `/home/malory' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: No password supplied
Retype new password:
No password has been supplied.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N]
Changing the user information for malory
Enter the new value, or press ENTER for the default
        Full Name []: Malory
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n]
info: Adding new user `malory' to supplemental / extra groups `users' ...
info: Adding user `malory' to group `users' ...
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 5: Creating user account for Malory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo adduser trent
info: Adding user `trent' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `trent' (1006) ...
info: Adding new user `trent' (1006) with group `trent (1006)' ...
info: Creating home directory `/home/trent' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: No password supplied
Retype new password:
No password has been supplied.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N]
Changing the user information for trent
Enter the new value, or press ENTER for the default
        Full Name []: Trent
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n]
info: Adding new user `trent' to supplemental / extra groups `users' ...
info: Adding user `trent' to group `users' ...
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 6: Creating user account for Trent

```
cyberstudent@CNIT270-Summer-2026:~$ sudo adduser walter
info: Adding user `walter' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `walter' (1007) ...
info: Adding new user `walter' (1007) with group `walter (1007)' ...
info: Creating home directory `/home/walter' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: No password supplied
Retype new password:
No password has been supplied.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N]
Changing the user information for walter
Enter the new value, or press ENTER for the default
        Full Name []: Walter
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n]
info: Adding new user `walter' to supplemental / extra groups `users' ...
info: Adding user `walter' to group `users' ...
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 7: Creating user account for Walter

```
cyberstudent@CNIT270-Summer-2026:~$ sudo adduser victor
info: Adding user `victor' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `victor' (1008) ...
info: Adding new user `victor' (1008) with group `victor (1008)' ...
info: Creating home directory `/home/victor' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: No password supplied
Retype new password:
No password has been supplied.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N]
Changing the user information for victor
Enter the new value, or press ENTER for the default
        Full Name []: Victor
        Room Number []:
        Work Phone []:
        Home Phone []: ]
        Other []:
Is the information correct? [Y/n]
info: Adding new user `victor' to supplemental / extra groups `users' ...
info: Adding user `victor' to group `users' ...
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 8: Creating user account for Victor

```
cyberstudent@CNIT270-Summer-2026:~$ sudo adduser hands
info: Adding user `hands' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `hands' (1009) ...
info: Adding new user `hands' (1009) with group `hands (1009)' ...
info: Creating home directory `/home/hands' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: No password supplied
Retype new password:
No password has been supplied.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N]
Changing the user information for hands
Enter the new value, or press ENTER for the default
        Full Name []: Hands
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n]
info: Adding new user `hands' to supplemental / extra groups `users' ...
info: Adding user `hands' to group `users' ...
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 9: Creating user account for Hands

The method to confirm that all users were created involves searching `/etc/passwd`. By running `grep '/home' /etc/passwd`, a filtered search of `/etc/passwd` outputs only user accounts with that have their own home directory in the `/home` folder (Linux Foundation, 2026). Figure 10 provides evidence of successful creation of all users.

1. Open **Terminal**
2. Type `grep 'home' /etc/passwd`

```
cyberstudent@CNIT270-Summer-2026:~$ grep 'home' /etc/passwd
syslog:x:104:110::/home/syslog:/usr/sbin/nologin
cups-pk-helper:x:113:120:user for cups-pk-helper service,,,:/home/cups-pk-helper:/usr/sbin/nologin
cyberstudent:x:1000:1000:cyberstudent,,,:/home/cyberstudent:/bin/bash
alice:x:1001:1001:Alice,,,:/home/alice:/bin/bash
bob:x:1002:1002:Bob,,,:/home/bob:/bin/bash
carol:x:1003:1003:Carol,,,:/home/carol:/bin/bash
eve:x:1004:1004:Eve,,,:/home/eve:/bin/bash
malory:x:1005:1005:Malory,,,:/home/malory:/bin/bash
trent:x:1006:1006:Trent,,,:/home/trent:/bin/bash
walter:x:1007:1007:Walter,,,:/home/walter:/bin/bash
victor:x:1008:1008:Victor,,,]:/home/victor:/bin/bash
hands:x:1009:1009:Hands,,,:/home/hands:/bin/bash
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 10: User creation audit

Certain users needed to be given administrative permissions to manage the system. Utilizing the method usermod -aG to modify a user's membership to a sudo level achieved this objective. This command adds a user to a specific group, in our case the sudo group. It grants privileges under the sudo role. However, it must be note, in order for the sudo permissions to go into effect, the user must log-out and log back in before their new access will take effect (SamTech, 2013). Figures 11, 12, and 13 provide evidence of the successful update of permissions for the respective users.

1. Opened **Terminal**
2. Typed `sudo usermod -aG sudo <user>`
3. Repeated for all users requiring sudo permissions

```
cyberstudent@CNIT270-Summer-2026:~$ sudo usermod -aG sudo alice
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 11: Providing Alice with sudo permissions

```
cyberstudent@CNIT270-Summer-2026:~$ sudo usermod -aG sudo bob
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 12: Providing Bob with sudo permissions

```
cyberstudent@CNIT270-Summer-2026:~$ sudo usermod -aG sudo hands
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 13: Providing Hands with sudo permissions

The method to confirm that the respective users were given administrative privileges is with the `getent` command. The `getent` command displays entries from system databases like users, groups, and hosts. To find all sudo or super users in Linux, the `getent` command is used as such: `getent group sudo` (Palani, 2025). Figure 14 provides evidence of the method used to audit that all respective users have administrative (sudo) privileges.

```
cyberstudent@CNIT270-Summer-2026:~$ getent group sudo
sudo:x:27:cyberstudent,alice,bob,hands
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 14: Audit of Alice, Bob, and Hands users having sudo permissions

When users were initially created, a standard empty password was used for all created users. This does not follow standard security requirements. Thus, `sudo passwd –expire <username>` was used. This command will make a user's password expire instantly. An expired password in Linux will force the user to change their password the next time they log in (kbmisc.com, 2022). Figures 15-23 showcase the method used to expire all users passwords immediately.

1. Open **Terminal**
2. Type sudo passwd --expire <username>
3. Press **enter**
4. Repeat for all users

```
cyberstudent@CNIT270-Summer-2026:~$ sudo passwd --expire alice
passwd: password changed.
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 15: Password expire for Alice

```
cyberstudent@CNIT270-Summer-2026:~$ sudo passwd --expire bob
passwd: password changed.
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 16: Password expire for Bob

```
cyberstudent@CNIT270-Summer-2026:~$ sudo passwd --expire carol
passwd: password changed.
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 17: Password expire for Carol

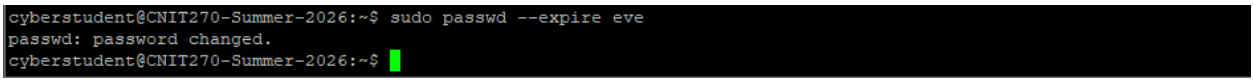


Figure 18: Password expire for Eve

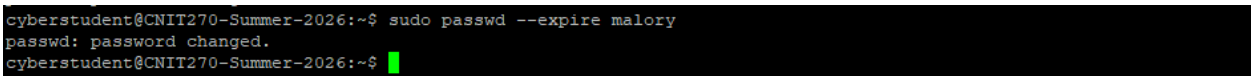


Figure 19: Password expire for Malory

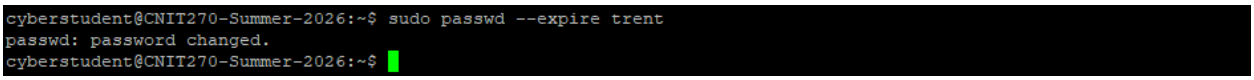


Figure 20: Password expire for Trent

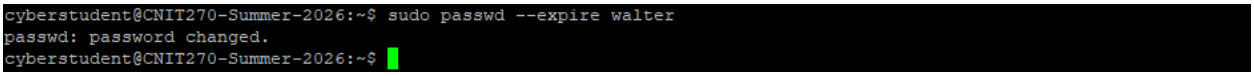


Figure 21: Password expire for Walter

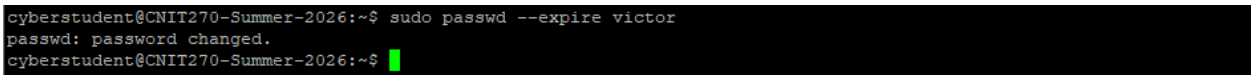


Figure 22: Password expire for Victor

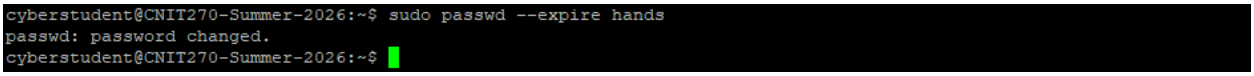


Figure 23: Password expire for Hands

To audit that the passwords for all users expired immediately, the `chage` command allows us to view the details of the password expiration date of a specific user. By running `chage` with the `-l` option, a list is outputted with details on the last password change, password expiry date, password inactive date, and more. To confirm if the password has been expired, the last password change, password expires, and password inactive tags should be valued as “password must be changed” (kbmisc.com, 2022). Figures 24-32 provide evidence that all user accounts had their passwords expired, using the `chage` method.

1. Open **Terminal**
2. Type `sudo chage -l <username>`
3. Repeat for all users

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chage -l alice
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 24: Password expire audit for Alice

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chage -l bob
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 25: Password expire audit for Bob

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chage -l carol
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 26: Password expire audit for Carol

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chage -l eve
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 27: Password expire audit for Eve

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chage -l malory
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 28: Password expire audit for Malory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chage -l trent
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 29: Password expire audit for Trent

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chage -l walter
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 30: Password expire audit for Walter

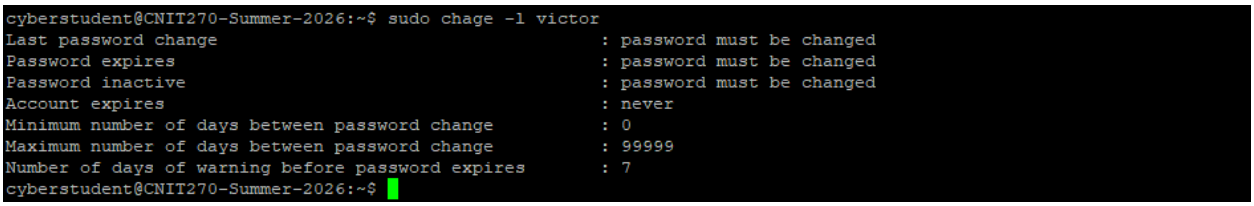


Figure 31: Password expire audit for Victor

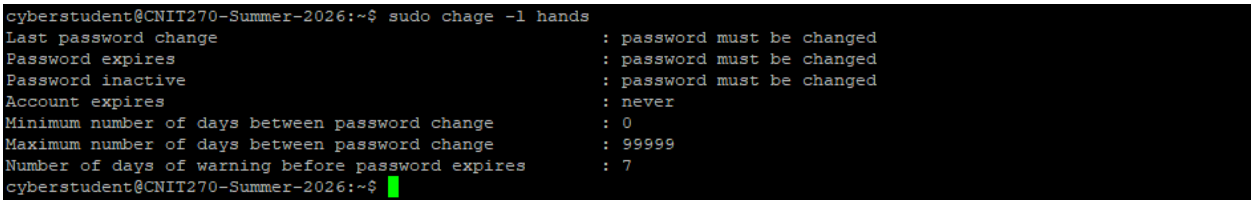


Figure 32: Password expire audit for Hands

Trent works in R&D and does not require administrative privileges. The principle of least privilege states that users should only be granted the minimum level of access necessary to perform their job functions (Stallings & Brown, 2018). R&D employees need access to project files and collaboration tools, but granting system-level administrative access goes beyond what is operationally needed. In the case that Trent's account was compromised, an attacker with admin privileges could alter system configurations, access all user data, install malicious software.

Malory works in Accounting and does not require administrative privileges. Granting administrative access to Malory violates the principle of least privilege and undermines non-repudiation, which requires that actions on a system be traceable to a specific, accountable individual (Stallings & Brown, 2018). With admin rights, it would become difficult to determine whether changes to financial records were authorized, creating audit and compliance risks. Restricting Malory to standard user access limits the potential damage of both accidental misuse and malicious insider activity.

Permission Bits Review

This section provides two tables as reference to the Linux regular permission bits, and

Linux special permission bits. These two tables are meant to serve as a reference, or key, to future IT employees interpreting permission listings. The regular permission bits follow the POSIX standard for discretionary access control as defined by The Open Group (2018). The special permission bits, including SUID, SGID, and the sticky bit, are documented in the Linux kernel interface (Linux man-pages project, 2023).

Table 1: Linux Regular Permission Bits and Descriptions

Binary Bit Permutations	**Octal Bits**	**Qualitative Description**
000	0	No read, write, or execute. No access
001	1	Execute only. User can run files as programs. User will not be able to read contents, or write to contents.
010	2	Write only. User can modify files and programs but can not execute nor read them.
011	3	Write and execute files. User cannot read files, but can modify and run them.
100	4	Read only.
101	5	Read and execute files.
110	6	Read and write files.
111	7	Read, write, and execute files. User has full permissions.

Table 2: Linux Special Permission Bits and Descriptions

Binary Bit Permutations	**Octal Bits**	**Qualitative Description**
000	0	No special bits set
001	1	Sticky bit. Only file owner can delete/rename the file

010	2	SGID, files created in directory inherit the group; executable runs with group's privileges
011	3	SGID + sticky bit
100	4	SUID, executable runs with the file owner's privileges
101	5	SUID + Sticky Bit
110	6	SUID + SGID
111	7	SUID + SGID + Sticky bit

Creating Groups

With multiple users within the organization, across many departments, user groups were created for each department using the `groupadd` command. The command `usermod -aG` assigns a user to a specific department (GeeksforGeeks, 2026). Each respective user (employee) was added to their respective department, or group. Figures 33-39 provide evidence to the method used to create all groups. Figure 40 provides evidence of users being added to their respective groups.

1. Open **Terminal**
2. Type sudo `groupadd <department>`
3. Repeat for every department
4. Type sudo `usermod -aG <department> <user>`
5. Repeat for every user

```
cyberstudent@CNIT270-Summer-2026:~$ sudo groupadd sales
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 33: Sales group creation

```
cyberstudent@CNIT270-Summer-2026:~$ sudo groupadd it
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 34: IT group creation

```
cyberstudent@CNIT270-Summer-2026:~$ sudo groupadd accounting
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 35: Accounting group creation

```
cyberstudent@CNIT270-Summer-2026:~$ sudo groupadd rd
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 36: R&D group creation

```
cyberstudent@CNIT270-Summer-2026:~$ sudo groupadd reception
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 37: Reception group creation

```
cyberstudent@CNIT270-Summer-2026:~$ sudo groupadd customerservice
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 38: Customer service group creation

```
cyberstudent@CNIT270-Summer-2026:~$ sudo groupadd remoteusers
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 39: Remote users group creation

```
cyberstudent@CNIT270-Summer-2026:~$ getent group sales it acccouting rd reception customerservice
sales:x:1010:carol
it:x:1011:alice,bob
rd:x:1013:trent,walter
reception:x:1014:eve
customerservice:x:1015:victor
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 40: Evidence users added to correct groups

With each group created, a fileshare was created for each group. Each with a test file to ensure proper operation. The fileshare can be accessed by all members of the group. This is a simple way to share files within the group it is assigned to. It is important to note that users can only access the fileshare of their own group, and not others (Beck et al., 2012). The /shares directory was created at the root level, adding only one folder to /. Each group's share is a subdirectory of /shares, which is owned by root. This meets all directory structure requirements. Figures 41-47 provide evidence of how the directories were created.

1. Opened **Terminal**
2. Type sudo mkdir /shares
3. Type sudo mkdir /shares/<department>
4. Repeat step 3 for each department

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /shares
[sudo] password for cyberstudent:
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 41: Creation of /shares

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /shares/sales
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 42: Creation of /shares/sales

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /shares/accounting
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 43: Creation of /shares/accounting

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /shares/rd
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 44: Creation of /shares/rd

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /shares/reception
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 45: Creation of /shares/reception

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /shares/customerservice
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 46: Creation of /shares/customerservice

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /shares/it
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 47: Creation of /shares/it

Once group shared directories were created for each department, permissions had to be set for each. Following the lab manual, group ownership of each directory was to be created. Then, specific read, write, and execute permissions were to be set for each directory. `sudo chown` is used to change the access rights of a directory or file (Ludosanu, 2024). Following the manual, the command sudo chown root:<groupname> /shares/<groupname> was used to assign group ownership of each respective directory. Figures 48-53 provide evidence of the method used to set group ownership of each directory.

1. Open **Terminal**
2. Type `sudo chown root:<department> /shares/<department>`
3. Repeat for all department directories

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chown root:sales /shares/sales
[sudo] password for cyberstudent:
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 48: Set sales group ownership

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chown root:it /shares/it
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 49: Set IT group ownership

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chown root:accounting /shares/accounting
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 50: Set accounting group ownership

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chown root:rd /shares/rd
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 51: Set R&D group ownership

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chown root:reception /shares/reception
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 52: Set reception group ownership

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chown root:customerservice /shares/customerservice
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 53: Set customer service group ownership

After, `sudo chmod 3770 /shares/<groupname>` is used to set permissions for each directory. Permission 3770 uses octal 3 for special bits (SGID + sticky), 7 for owner (full access), 7 for group (full access), and 0 for others (no access) (J., 2026). Figures 54-59 provide evidence of the method used to set permissions.

1. Open **Terminal**
2. Type `sudo chmod 3770 /shares/<groupname>`
3. Repeat for all department directories

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chmod 3770 /shares/sales
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 54: Set 3770 special permission for sales directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chmod 3770 /shares/it
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 55: Set 3770 special permission for IT directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chmod 3770 /shares/accounting
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 56: Set 3770 special permission for accounting directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chmod 3770 /shares/rd
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 57: Set 3770 special permission for R&D directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chmod 3770 /shares/reception
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 58: Set 3770 special permission for reception directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chmod 3770 /shares/customerservice
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 59: Set 3770 special permission for customer service directory

Once group shared directories were created for each department, the test files were created within each. The command `sudo touch` was used to create a file within each respective directory. The fundamental syntax for the touch command is `touch <file or`

`directory name>` (Dancuk, 2025). The naming of the files was intentional to maintain neatness and to avoid Linux syntax errors of using spaces. Files were named as department_test.txt. Figures 60-65 provide evidence of the method used to create test files within each shared directory for each department. Figure 66 provides evidence of the correct file structure utilizing the `tree` command. The tree command in Linux displays the directory structure in a hierarchical, tree-like format. This provides a clear visual representation of files and subdirectories (GeeksforGeeks, 2025).

1. Opened **Terminal**
2. Type sudo -u <user> touch /shares/<department>/<department>_test.txt
3. Repeat for all users with their respective directories and departments
4. Type sudo tree /shares

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u carol touch /shares/sales/sales_test.txt
[sudo] password for cyberstudent:
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 60: Create sample test file in sales directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u alice touch /shares/it/it_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 61: Create sample test file in IT directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u malory touch /shares/accounting/accounting_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 62: Create sample test file in accounting directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u trent touch /shares/rd/rd_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 63: Create sample test file in R&D directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u eve touch /shares/reception/reception_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 64: Create sample test file in reception directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u victor touch /shares/customerservice/customerservice_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 65: Create sample test file in customer service directory

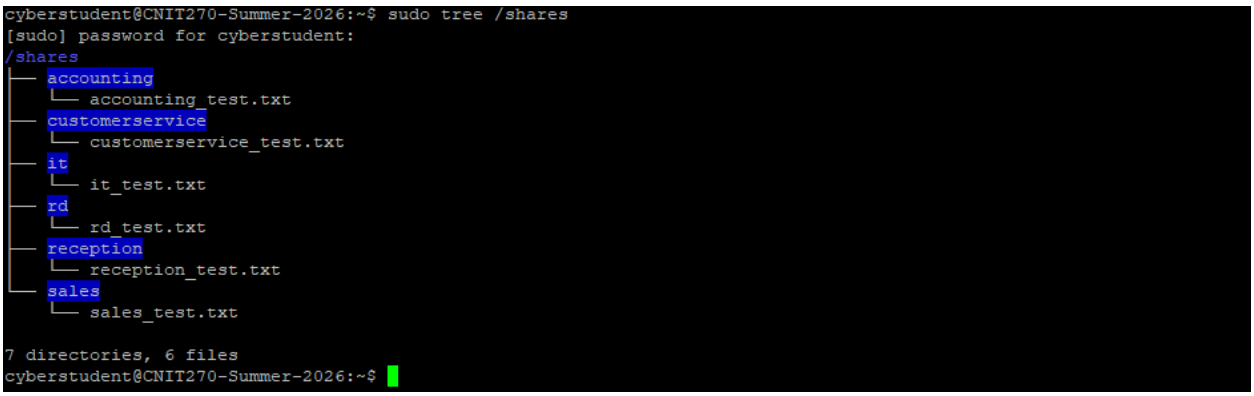


Figure 66: Evidence of correct file structure

It is imperative to audit that all steps are followed according to the manual. Below states the steps to audit that all directories have the correct permissions and groups attached. Figure 66 provides evidence that all directories were created, and with the correct and reasonable permissions. Figures 54-59 provide evidence of the method used to set these permissions. To audit and confirm that the current permissions were set to each directory, `sudo ls -la /shares/` was used. Then `sudo ls -la /shares/<department>/` was used to check permissions within each directory. The ls command lists the contents of a directory object in a work area. By default, the output consists of a list of objects and their associated projections in the file system (IBM Corporation, 2021).

1. Open **Terminal**
2. Type sudo ls -la /shares/
3. Type sudo ls -la /shares/<department>/
4. Repeat step 3 for each directory

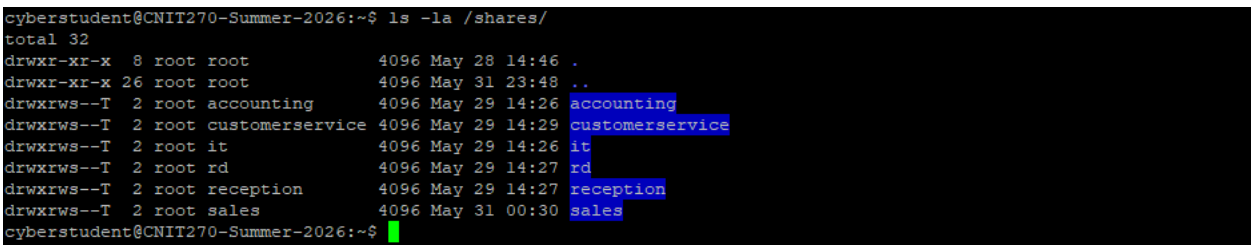


Figure 67: Execution to show audit of permissions within /shares/ directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo ls -la /shares/accounting
[sudo] password for cyberstudent:
total 8
drwxrws--T 2 root   accounting 4096 May 29 14:26 .
drwxr-xr-x 8 root   root       4096 May 28 14:46 ..
-rw-rw-r-- 1 malory accounting    0 May 29 14:26 accounting_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 68: Execution to show audit permissions within accounting directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo ls -la /shares/customerservice
total 8
drwxrws--T 2 root   customerservice 4096 May 29 14:29 .
drwxr-xr-x 8 root   root           4096 May 28 14:46 ..
-rw-rw-r-- 1 victor customerservice    0 May 29 14:29 customerservice_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 69: Execution to show audit permissions within customerservice directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo ls -la /shares/it
total 8
drwxrws--T 2 root  it   4096 May 29 14:26 .
drwxr-xr-x 8 root  root 4096 May 28 14:46 ..
-rw-rw-r-- 1 alice it      0 May 29 14:26 it_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 70: Execution to show audit permissions withing IT directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo ls -la /shares/rd
total 8
drwxrws--T 2 root  rd   4096 May 29 14:27 .
drwxr-xr-x 8 root  root 4096 May 28 14:46 ..
-rw-rw-r-- 1 trent rd      0 May 29 14:27 rd_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 71: Execution to show audit permissions within R&D directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo ls -la /shares/reception
total 8
drwxrws--T 2 root reception 4096 May 29 14:27 .
drwxr-xr-x 8 root root      4096 May 28 14:46 ..
-rw-rw-r-- 1 eve  reception    0 May 29 14:27 reception_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 72: Execution to show audit permissions within reception directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo ls -la /shares/sales
total 8
drwxrws--T 2 root  sales 4096 May 31 00:30 .
drwxr-xr-x 8 root  root  4096 May 28 14:46 ..
-rw-rw-r-- 1 hands sales    0 May 31 00:23 handsfile.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 73: Execution to show audit permissions withing sales directory

Group Audit

To confirm that the group file share permissions are configured correctly, an audit was conducted. This audit follows the principle of least privilege by verifying that authorized users can perform only the actions they are supposed to, and that unauthorized users are blocked entirely. The audit is organized into three categories: authorized user use cases, authorized user misuse cases, and unauthorized user misuse cases (Stallings & Brown, 2018).

Audit Method – Authorized User Use Cases

An authorized user is defined as a member of the group that owns a given file share

directory. The following use cases confirm that group members retain the access they need to do their jobs.

Use Case 1 - Create a new file

A group member should be able to create new files in their group's share directory. This is a fundamental requirement for collaboration — employees need to be able to add documents, reports, and working files to the shared space. Figure 74 shows the expected result where the command succeeds with no error. The file is created by Carol, and the group automatically sets to sales.

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u carol touch /shares/sales/newfile.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 74: New file created by Carol in sales directory

Use Case 2 - Read a file

A group member should be able to read any file within their group's share, regardless of who created it. This supports collaboration by allowing all members to view shared documents. Figure 75 shows the expected result where the command succeeds with no error. Thus, the permissions to read files within the respective sales directory for Carol is correct.

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u carol cat /shares/sales/sales_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 75: File read by Carol in sales directory

Use Case 3 - Modify a file the user owns

A group member should be able to edit files that they personally created within the share. This allows employees to update and maintain their own work. Figure 76 shows the nano command succeeding without error. Thus, the permissions to write files within the respective sales directory as Carol is correct. Figure 77 shows the view when editing the file.

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u carol nano /shares/sales/newfile.txt
[sudo] password for cyberstudent:
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 76: Successful run of nano on text file in sales directory by Carol

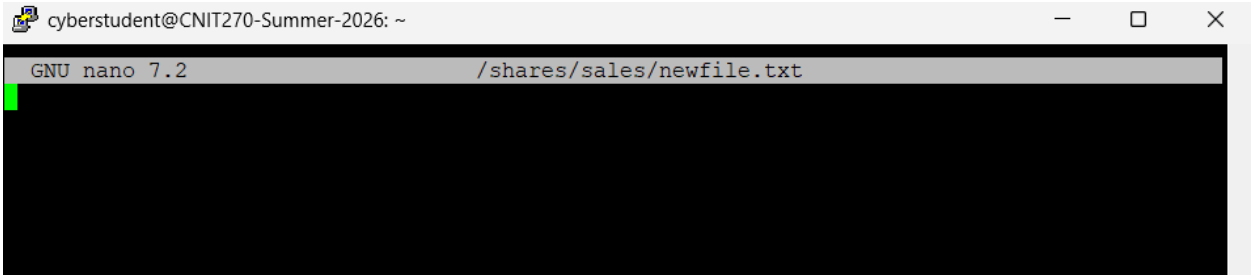


Figure 77: Nano view after running command showcased in Figure 69

Use Case 4 - Delete a file the user owns

A group member should be able to delete files they created. The sticky bit ensures this is limited to only their own files. This gives employees control over their own work without allowing them to disrupt others. Figure 78 shows the rm command running successfully without any error.

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u carol rm /shares/sales/newfile.txt
[sudo] password for cyberstudent:
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 78: Successful run of rm on text file in sales directory for user Carol

Use Case 5 - List directory contents

A group member should be able to view what files exist within their respective directory. Figure 79 shows the successful running of the ls command for Carol in the sales directory. Directory listing is displayed successfully showing all files in the share. Thus, showcasing that the permissions for Carol to view the directory are correct.

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u carol ls -la /shares/sales
total 8
drwxrws--T 2 root  sales 4096 May 30 23:22 .
drwxr-xr-x 8 root  root  4096 May 28 14:46 ..
-rw-rw-r-- 1 carol sales    0 May 29 14:25 sales_test.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 79: Successful run of ls on sales directory for user Carol

Audit Method - Authorized User Mis-Cases

These audit tests confirm that group members cannot perform actions that they do not have permission for.

Misuse Case 1 - Delete a file owned by another group member

Even though carol is a member of the sales group, the sticky bit (1 in the special bits) on

the directory prevents her from deleting files she does not own. This protects employees' work from being deleted by peers. Figure 80 shows a test file created named handsfile.txt by a different user, Hands, in the sales directory. Figure 81 showcased how an attempt by Carol to delete that file will fail.

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u hands touch /shares/sales/handsfile.txt
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 80: Test file handsfile.txt created in sales directory under user Hands

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u carol rm /shares/sales/handsfile.txt
rm: cannot remove '/shares/sales/handsfile.txt': Operation not permitted
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 81: Attempt by Carol to delete handsfile.txt fails

Misuse Case 2 - Delete the entire group share directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u carol rm -r /shares/sales
rm: cannot remove '/shares/sales/handsfile.txt': Operation not permitted
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 82: Attempt by Carol to delete sales directory fails

Audit Method - Unauthorized User Misuse Cases

An unauthorized user is defined as any user who is not a member of the group that owns the share. These tests confirm that the 0 in the "other" permission position blocks all outside access, enforcing strict compartmentalization between departments.

Misuse Case 4 - Non-member attempts to list directory contents

A user from a different department (e.g., victor from Customer Service) should not be able to see what files exist in the sales share. Figure 82 showcases an error when attempting to have Victor

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u victor ls /shares/sales
[sudo] password for cyberstudent:
ls: cannot open directory '/shares/sales': Permission denied
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 83: Attempt by Victor to list sales directory contents

Misuse Case 5 - Non-member attempts to read a file

Victor should not be able to read the contents of any file in the sales share, protecting confidential sales data from other departments.

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u victor cat /shares/sales/sales_test.txt
cat: /shares/sales/sales_test.txt: Permission denied
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 84: Attempt by Victor to read a file in the sales directory

Misuse Case 6 - Non-member attempts to create a file

Victor should not be able to write new files into the sales share. Allowing this would let any user pollute or tamper with another group's working directory.

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u victor touch /shares/sales/instrusion.txt
touch: cannot touch '/shares/sales/instrusion.txt': Permission denied
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 85: Attempt by Victor to write a new file into the sales directory

Misuse Case 7 - Non-member attempts to delete a file

Victor should not be able to delete any files from the sales share, preventing cross-department data destruction.

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u victor rm /shares/sales/sales_test_txt
rm: cannot remove '/shares/sales/sales_test_txt': Permission denied
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 86: Attempt by Victor to delete a file in the sales directory

Misuse Case 8 - Non-member attempts to delete the share directory

No non-administrative user from outside the group should be able to remove the directory entirely.

```
cyberstudent@CNIT270-Summer-2026:~$ sudo -u victor rm -r /shares/sales
rm: cannot remove '/shares/sales': Permission denied
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 87: Attempt by Victor to delete the sales directory

Root Account

Disable Root Login Method

The root account was to be disabled using the `passwd -l command`. The password was then locked for the root account. This made it so the root user could not authenticate with a password (IBM Corporation, 2024). Figure 88 shows successful completion of the command with the output stating that the password was changed.

1. Open **Terminal**

2. Type sudo passwd -l root



Figure 88: Sudo passwd -l root execution

The /etc/shadow was audited to check the hash. The `sudo grep root /etc/shadow` command extracts the security and password-hashing information for the root user from the /etc/shadow file. Because this file contains sensitive password hashes, it is highly restricted and requires administrative (sudo) privileges to read. The ! prefix in the shadow password field indicates the account is locked/disabled (Dhandala, 2026). Figure 89 provides the method used to view the hash for /etc/shadow. Figure 90 shows the method used to create a new password for the root. Figure 91 shows the method used in Figure 89, executed again to show the changed hash after running the command in Figure 90.

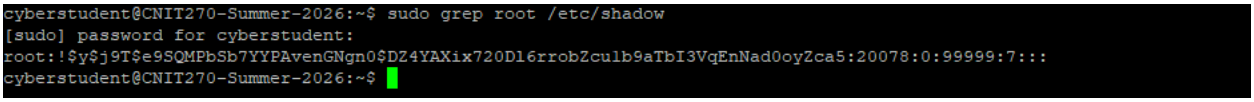


Figure 89: Completion of sudo grep root /etc/shadow

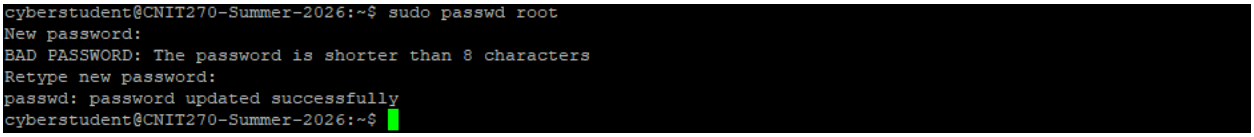


Figure 90: Create new password for root

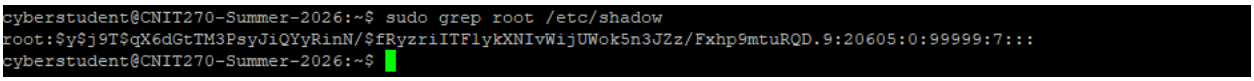


Figure 91: Check new hash for root password

/etc & /etc/shadow

Following the manual and Hands request. Tables 3 and 4 provide a key to the format of an entry in /etc/passwd, and /etc/shadow.

Table 3: Reference table for the format of an entry in /etc/passwd

Field	**Example**	**Description**

All Fields	alice:x:1001:1001:Alice Smith:/home/alice:/bin/bash	The complete entry for a single user in /etc/passwd (Gite, 2025b).
Username	alice	The username used for login (Gite, 2025b).
Password	x	The password (denoted by x) is encrypted and stored in /etc/shadow (Gite, 2025b).
Numerical User ID	1001	The user ID associated with the account (Gite, 2025b).
Numerical Group ID	1001	The group ID associated with the user's group (Gite, 2025b).
Comment Field	Alice Smith	A comment field storing extra information about the user (full name, phone number, etc..) (Gite, 2025b).
Home Directory	/home/alice	The absolute path of the home directory of the user (Gite, 2025b).
Command/shell	/bin/bash	The absolute path of the command/shell interpreter used at logon (Gite, 2025b).

Table 4: Reference table for the format of an entry in /etc/shadow

Field	**Example**	**Description**

All Fields	alice:6abc123hashedpassword:19793:0:99999:7:1::	The complete entry for a single user in /etc/passwd (Gite, 2025a).
Username	alice	The valid account name that exists on the system used for logon (Gite, 2025a).
Password	6abc123hashedpassword	The encrypted and hashed password (Gite, 2025a).
Last password change	19793	The date of last password change (expressed as # of days since 1/1/1970) (Gite, 2025a).
Minimum	0	The minimum amount of time required between password changes (Gite, 2025a).
Maximum	99999	The maximum amount of time a single password can exist before it must be changed (Gite, 2025a).

Warn	7	The amount of time before a password will expire (Gite, 2025a).
Inactive	1	The amount of time after a password expiration where the password will still be accepted (Gite, 2025a).
Expire	{empty}	The expiration date of the account (expressed as # of days since 1/1/1970) (Gite, 2025a).

Create Character Devices

Certain remote users need to access the system over Secure Shell (SSH). For security purposes, a chroot (change + root) jail was implemented, ensuring that the actual filesystem was protected when over a remote connection, while allowing remote users to access only the tools needed. The first step was to create the file structure, including the folders to contain the required binaries and system files. Figures 92-97 provide evidence to the method used to create the directory requested in the lab manual.

1. Open **Terminal**
2. Type sudo mkdir /detention
3. Type sudo mkdir /detention/dev
4. Type sudo mkdir /detention/etc

5. Type sudo mkdir /detention/lib
6. Type sudo mkdir /detention/usr
7. Type sudo mkdir /detention/bin
8. Type sudo mkdir /detention/usr/bin
9. Type sudo tree /detention

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /detention
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 92: Creating detention directory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /detention/dev
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 93: Creating dev subdirectory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /detention/etc
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 94: Creating etc subdirectory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /detention/lib
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 95: Creating lib subdirectory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /detention/usr
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 96: Creating usr subdirectory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mkdir /detention/usr/bin
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 97: Creating bin subdirectory under usr subdirectory

```
cyberstudent@CNIT270-Summer-2026:~$ sudo tree /detention
/detention
├── bin
├── dev
├── etc
├── lib
└── usr
    └── bin

7 directories, 0 files
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 98: Proof of created directories

Once the file system for the Chroot jail is configred, a Remote Users group is added to the system. This allows for only specific users to be given remote access. Below, only the required users were added to the new remoteusers group.

1. Open **Terminal**
2. Type `sudo usermod -aG remoteusers <user>`
3. Repeat for every required user

```
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$ sudo usermod -aG remoteusers carol
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$
```

Figure 99: Add Carol to remoteusers group

```
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$ sudo usermod -aG remoteusers alice
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$
```

Figure 100: Add Alice to remoteusers group

Necessary device files need to be created. This is to support normal system operations inside the jail environment. In Linux environments, /dev/random acts as a character device designed to distribute high-grade random data to software applications. To safeguard the integrity of the operating system's entropy pool from malicious or corrupted modifications, it is configured exclusively for public consumption using a read-only permission structure like mode 444. Conversely, /dev/tty functions as the bridge to a process's active terminal window. Because terminal interactions necessitate simultaneous user input and application output, it requires bidirectional access. This makes a read-and-write permission mode like 666 essential for everyday system communication (Krrisik 2024).

1. Opened **Terminal**
2. Type `sudo mknod -m 666 /detention/dev/null c 1 3`
3. Type `sudo mknod -m 622 /detention/dev/zero c 1 5`
4. Type `sudo mknod -m 666 /detention/dev/tty c 5 0`
5. Type `sudo mknod -m 444 /detention/dev/random c 1 8`
6. Type `sudo mknod -m 444 /detention/dev/urandom c 1 9`
7. Type `sudo chown root:tty /detention/dev/tty`

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mknod -m 666 /detention/dev/null c 1 3
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 101: Create null device using mknod

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mknod -m 622 /detention/dev/zero c 1 5
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 102: Create zero device using mknod

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mknod -m 666 /detention/dev/tty c 5 0
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 103: Create tty device using mknod

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mknod -m 444 /detention/dev/random c 1 8
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 104: Create random device using mknod

```
cyberstudent@CNIT270-Summer-2026:~$ sudo mknod -m 444 /detention/dev/urandom c 1 9
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 105: Create random device file using mknod

```
cyberstudent@CNIT270-Summer-2026:~$ sudo chown root:tty /detention/dev/tty
cyberstudent@CNIT270-Summer-2026:~$
```

Figure 106: Change ownership of tty file

After the file structure is set, users are added, and character devices configured, the physical binaries need to be copied from the physical system root to the detention. The process used the directories created when the file structure for the jail was implemented (Newell, 2025).

1. Open **Terminal**
2. Type cd /detention/etc
3. Type sudo cp /etc/ld.so.cache .
4. Type sudo cp /etc/ld,so.conf .
5. Type sudo cp /etc/nsswitch.conf .
6. Type sudo cp /etc/hosts .
7. Type cd /detention/usr/bin
8. Type sudo cp /bin/ls .
9. Type sudo cp /bin/bash .

```
cyberstudent@CNIT270-Summer-2026:~$ cd /detention/etc
cyberstudent@CNIT270-Summer-2026:/detention/etc$
```

Figure 107: Change to the /detention/etc directory

```
cyberstudent@CNIT270-Summer-2026:/detention/etc$ sudo cp /etc/ld.so.cache .
cyberstudent@CNIT270-Summer-2026:/detention/etc$
```

Figure 108: Copy the ld.so.cache file to current directory

```
cyberstudent@CNIT270-Summer-2026:/detention/etc$ sudo cp /etc/ld.so.conf .
cyberstudent@CNIT270-Summer-2026:/detention/etc$
```

Figure 109: Copy the ld.so.conf file to the current directory

```
cyberstudent@CNIT270-Summer-2026:/detention/etc$ sudo cp /etc/nsswitch.conf .
cyberstudent@CNIT270-Summer-2026:/detention/etc$
```

Figure 110: Copy the nsswitch file to the current directory

```
cyberstudent@CNIT270-Summer-2026:/detention/etc$ sudo cp /etc/hosts .
cyberstudent@CNIT270-Summer-2026:/detention/etc$
```

Figure 111: Copy the hosts file to the current directory

```
cyberstudent@CNIT270-Summer-2026:/detention/etc$ cd /detention/usr/bin
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$
```

Figure 112: Change to the /detention/usr/bin directory

```
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$ sudo cp /bin/ls .
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$
```

Figure 113: Copy the ls file to the current directory

```
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$ sudo cp /bin/bash .
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$
```

Figure 114: Copy the bash file to the current directory

```
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$ sudo cp /home/cyberstudent/l2chroot .
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$
```

Figure 115: Copy the l2chroot directory to the current directory

```
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$ sudo chmod 755 l2chroot
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$
```

Figure 116: Set permission of l2chroot to 755

```
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$ ldd /bin/ls
        linux-vdso.so.1 (0x00007ffd7a1c1000)
        libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x0000792f8b795000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000792f8b400000)
        libpcre2-8.so.0 => /lib/x86_64-linux-gnu/libpcre2-8.so.0 (0x0000792f8b6fb000)
        /lib64/ld-linux-x86-64.so.2 (0x0000792f8b7fb000)
cyberstudent@CNIT270-Summer-2026:/detention/usr/bin$
```

Figure 117: List dynamic library dependecies of the ls binary

For the remote users to access the system over SSH, the SSH config file needs to be altered. The remoteusers group needs to be added for SSH to recognize the users. This process also includes giving the directory of the chroot jail. This allows for the system to point SSH users to the correct place. Verbose logging must also be enabled, so that administrators received more information about login processes when they occur (Newell, 2025).

1. Opened **Terminal**
2. Typed `sudo nano /etc/ssh/sshd_config`
3. Changed LogLevel INFO to LogLevel VERBOSE
4. Typed `Match group remoteusers` (at bottom of file)
5. Typed `ChrootDirectory /detention/` (below Step 4)

6. Pressed Ctrl + X | Enter

7. Typed `sudo systemctl restart ssh`

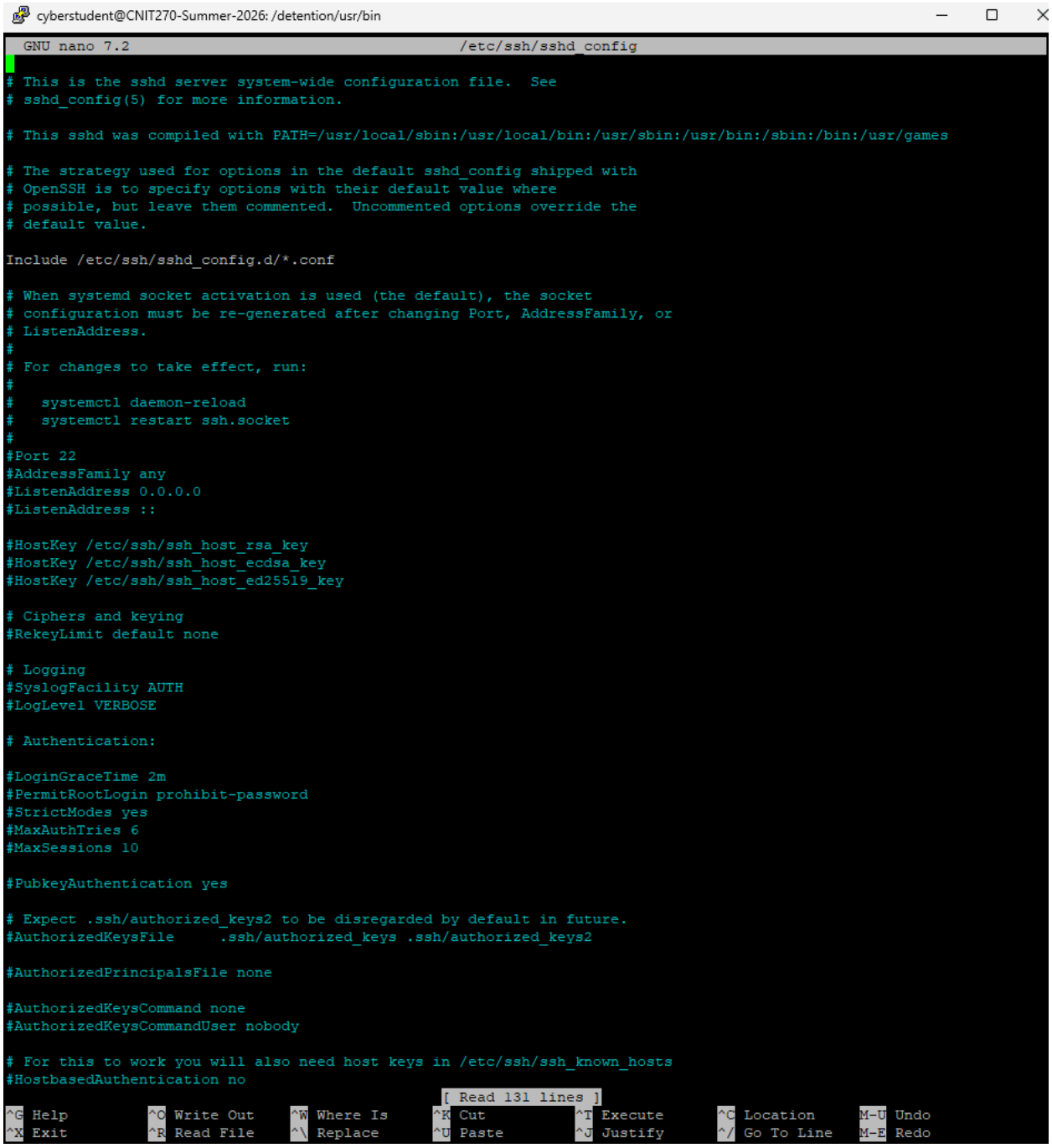


Figure 118: VERBOSE added

ACCESS CONTROL

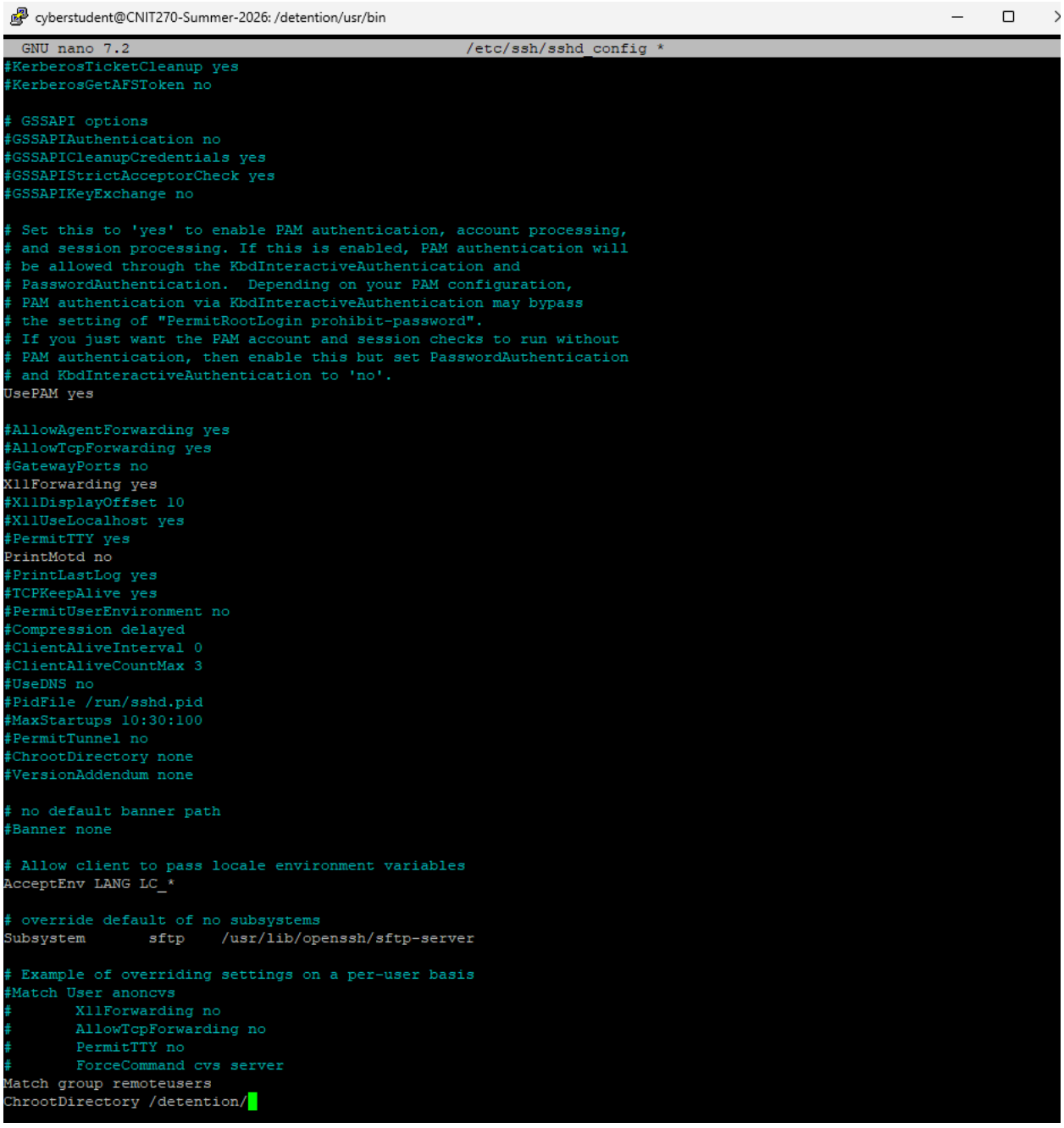


Figure 119: SSH Config with remote users and chroot directory

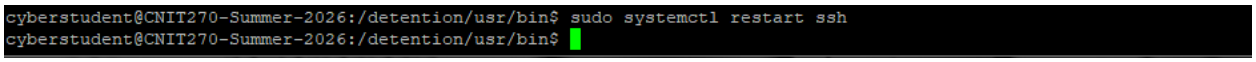


Figure 111: Restarting SSH

Chroot Jail: Testing

An SSH login was performed to confirm chroot jail functionality. This was done along with commands like ls and tree, which validated both the jail working, and that the required binaries were copied over from the root filesystem. SSH can be done from your terimal or in

PuTTY. After entering the appropriate IP address, type the users username, followed by the password to login. It is important to ensure that the connection type is set to SSH (Port 22) before opening the terminal built into PuTTY (Built by Pete, 2022). Figure 120 provides evidence of the successful SSH login as the user alice.

1. Open **Putty**
2. Type IP
3. Press open
4. Enter alice
5. Enter alice password
6. Type `ls`
7. Type `tree`



Figure 120: SSH login as user alice



Figure 121: ls command ran as alice



Figure 122: tree command ran as alice

Chroot Jail: Making the Jail Nice

To test the user confinement and confirm the restricted file structure, the following commands were run. Specifically, these commands show the necessary directory trees for

individual user spaces and departmental shared resources within the root /detention directory.

The command `mkdir`, creates a new directory. This is the core utility command in Linux to generate a new directory node inside the filesystem (IEEE Computer Society, 2016). -p is a modifier. -p allows for the creation of a nested parent directory (IEE Computer Society, 2016).

1. Opened **Terminal**
2. Type sudo mkdir -p /detention/home/alice
3. Type sudo mkdir -p /detention/home/carol
4. Type sudo mkdir -p /detention/shares/sales
5. Type sudo mkdir -p /detention/shares/it

```
cyberstudent@CNIT270-Summer-2026:/$ sudo mkdir -p /detention/home/alice
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 123: Create alice home directory inside detention

```
cyberstudent@CNIT270-Summer-2026:/$ sudo mkdir -p /detention/home/carol
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 124: Create carol home directory inside detention

```
cyberstudent@CNIT270-Summer-2026:/$ sudo mkdir -p /detention/shares/sales
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 125: Create sales directory inside detention

```
cyberstudent@CNIT270-Summer-2026:/$ sudo mkdir -p /detention/shares/it
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 126: Create IT directory inside detention

The manual requested that controlled acces is provided to existing host directories. Bind mounts are to be temporarily configured in the chroot jail. This allows for directories to be created for users, access to their respective files shares (Newell, 2025).

1. Opened **Terminal**
2. Type `sudo mount --bind home/alice /detention/home/alice`
3. Type `sudo mount --bind /home/carol/detention/home/carol`
4. Type `--bind /shares/it /detention/shares/it`
5. Type `--bind /shares/sales /detention/shares/sales`
6. Type `mount | grep detention`

```
cyberstudent@CNIT270-Summer-2026:/$ sudo mount --bind /home/alice /detention/home/alice
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 127: Mount the home directory for alice into chroot jail

```
cyberstudent@CNIT270-Summer-2026:/$ sudo mount --bind /home/carol /detention/home/carol
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 128: Mount the home directory for carol into the chroot jail

```
cyberstudent@CNIT270-Summer-2026:/$ sudo mount --bind /shares/it /detention/shares/it
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 129: Mount the shared IT directory to the chroot jail

```
cyberstudent@CNIT270-Summer-2026:/$ sudo mount --bind /shares/sales /detention/shares/sales
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 130: Mount the shared sales directory to the chroot jail

```
cyberstudent@CNIT270-Summer-2026:/$ mount | grep detention
/dev/sda2 on /detention/home/alice type ext4 (rw,relatime,errors=remount-ro)
/dev/sda2 on /detention/home/carol type ext4 (rw,relatime,errors=remount-ro)
/dev/sda2 on /detention/shares/it type ext4 (rw,relatime,errors=remount-ro)
/dev/sda2 on /detention/shares/sales type ext4 (rw,relatime,errors=remount-ro)
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 131: Verifications of binds

Tools and utilities are then to be added to the chroot jail. The required tools were copied for proper functionality within the environment. These contained binaries that are imperative for system function. For example, the vim text editor, which would be used by the remote users within the jail. The which command searches the systems PATH evironment variable to find the absolute file path of an executable. This revealsthat both tail and vim are located in /usr/bin (Newll, 2025).

```
cyberstudent@CNIT270-Summer-2026:/$ which tail
/usr/bin/tail
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 132: Locate tail

```
cyberstudent@CNIT270-Summer-2026:/$ which vim
/usr/bin/vim
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 133: Locate vim

```
cyberstudent@CNIT270-Summer-2026:/$ sudo cp /usr/bin/tail /detention/usr/bin
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 134: Copy /bin/tail to /detention/usr/bin

```
cyberstudent@CNIT270-Summer-2026:/$ sudo cp /usr/bin/vim /detention/usr/bin
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 135: Copy /bin/vim to detention/bin

```
cyberstudent@CNIT270-Summer-2026:/$ sudo /detention/usr/bin/l2chroot /usr/bin/tail
Copying shared files/libs to /jailed...
Copying /lib64/ld-linux-x86-64.so.2 /jailed/lib64...
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 136: Copy /bin/tail to /detention

```
cyberstudent@CNIT270-Summer-2026:/$ sudo /detention/usr/bin/l2chroot /usr/bin/vim
Copying shared files/libs to /jailed...
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 137: Copy /bin/vim to /detention

```
cyberstudent@CNIT270-Summer-2026:/$ ls -l /detention/usr/bin/tail
-rwxr-xr-x 1 root root 64032 May 31 23:47 /detention/usr/bin/tail
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 138: Permissions of /detention/usr/bin/tail

```
cyberstudent@CNIT270-Summer-2026:/$ ls -l /detention/usr/bin/vim
-rwxr-xr-x 1 root root 4126480 May 31 23:48 /detention/usr/bin/vim
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 139: Permissions of /detention/usr/bin/vim

After the above steps. It was then requested by the manual that the environment is updated with required system identity files to restore propery username display. This makes it easier for identification. Additionally, these steps help in eliminating the I Have No Name issue (Newell, 2025).

```
cyberstudent@CNIT270-Summer-2026:/$ sudo mkdir -p /detention/etc
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 140: Create etc directory with mkdir

```
cyberstudent@CNIT270-Summer-2026:/$ sudo cp /etc/passwd /detention/etc
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 141: Create passwd file to detention

```
cyberstudent@CNIT270-Summer-2026:/$ sudo cp /etc/group /detention/etc
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 142: Copygroup file to detention

```
cyberstudent@CNIT270-Summer-2026:/$ sudo cp /etc/nsswitch.conf /detention/etc
cyberstudent@CNIT270-Summer-2026:/$
```

Figure 143: Copy the nsswitch.conf file to detention

Chroot Jail: Nice Jail Testing

After configuring the jail environment, testing is then conducted to test user confinement and confirm the restricted filesystem structure (Newell, 2025).

1. Open **Terminal**
2. Type `ssh alice@<vm_ip>`
3. Type `ls /`
4. Type `ls /fileshare`
5. Type `echo “alice test 1”> alice_echo.txt`

6. Type `tail -n 2 alice_echo.text`

7. Type `touch /home/carol/fail_test.txt`



Figure 144: `ls /` command as user alice



Figure 145: Shares directory

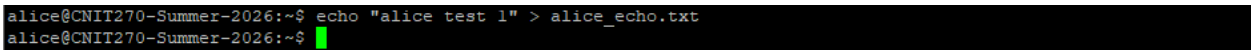


Figure 146: Alice test file creation



Figure 147: Attempt to show contents of alice test file as user alice

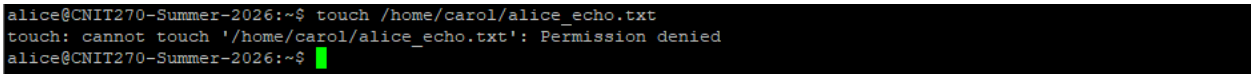


Figure 148: Failed attempt to access a file owned by user carol

BIBLIOGRAPHY

Beck, D., Turetta, A., & pabouk. (2012, February 7). Linux - Creating files and directories with a certain owner (user/group) while sudoing. Super User. https://superuser.com/questions/386885/creating-files-and-directories-with-a-certain-owner-user-group-while-sudoing

Built By Pete. (2022, March 2). *How to Install & use Putty to SSH on Windows* [Video]. YouTube. http://www.youtube.com/watch?v=x_P35yHG_Wk

Dancuk, M. (2025, December 19). How to use touch command in Linux (12 examples). phoenixNAP Knowledge Base. https://phoenixnap.com/kb/touch-command-in-linux

Dancuk, M. (2025, December 11). Linux adduser Command with Examples. phoenixNAP Knowledge Base. https://phoenixnap.com/kb/linux-adduser

Dhandala, N. (2026, March 2). *How to understand the /etc/passwd and /etc/shadow files on Ubuntu*. OneUptime. https://oneuptime.com/blog/post/2026-03-02-how-to-understand-the-etc-passwd-and-etc-shadow-files-on-ubuntu/view

IBM Corporation. (2021, March 5). *ls command*. IBM Documentation. https://www.ibm.com/docs/en/rational-synergy/7.2.1?topic=operations-ls-command

IBM Corporation. (2024, January 25). *Using the passwd command*. IBM Documentation. https://www.ibm.com/docs/en/zos/3.1.0?topic=commands-using-passwd-command

IEEE Computer Society. (2016). *The Open Group Base Specifications Issue 7 (POSIX.1-2008): mkdir — Make directories*. The Linux Manual Pages (Section 1). https://man7.org/linux/man-pages/man1/mkdir.1.html

J., R. (2026). *Special permissions with chmod 3770* [Lecture notes/Presentation slides]. Scribd. https://www.scribd.com/doc/7504017/LAMP-Chapter3

Kbmisc.com. (2022, October 14). How to make a user's password expire. https://kbmisc.com/blog/make-a-users-password-expire

Kerrisk, M. (Ed.). (2024). *random(4) — Linux manual page*. Linux man-pages project. https://man7.org/linux/man-pages/man4/random.4.html

Linux Foundation. (2026). *passwd(5): Password file* (Linux man-pages version 6.06). https://man7.org/linux/man-pages/man5/passwd.5.html

Linux man-pages project. (2023). *chmod(2) — Linux manual page*. https://man7.org/linux/man-pages/man2/chmod.2.html

Ludosanu, R. (2024, January 31). *What Does sudo chown Do?* Warp. https://www.warp.dev/terminus/sudo-chown

Newell, G. (2025, November 20). Taking a deeper dive into Linux chroot jails. *Red Hat Blog*. https://www.redhat.com/en/blog/deeper-linux-chroot-jails

Palani, S. (2025, August 14). *How to find all sudo users in Linux*. OSTechNix. https://ostechnix.com/find-sudo-users-linux-system/

SamTech. (2013, July 11). *Linux - Allow user to use sudo command has multiple ways, which one is better?* Super User. https://superuser.com/questions/618473/allow-user-to-use-sudo-command-has-multiple-ways-which-one-is-better

Stallings, W., & Brown, L. (2018). Computer security: Principles and practice (4th ed.). Pearson.

The Open Group. (2018). *The Open Group base specifications issue 7: File permissions*. The Open Group. https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap04.html